Complete MDP/SMDP Formulation (Option A)
Discrete High-Level Pursuit + Continuous Low-Level Control

Abstract

This document provides a complete MDP/SMDP formulation for the revised Option-A architecture: a *discrete high-level policy* selects pursuit macro-intents, while a *continuous low-level controller* (PID or continuous actor) executes physically feasible control under asymmetric pursuit-evasion dynamics. The formulation explicitly enforces the project asymmetry requirement $v^P_{\max} < v^E_{\max}$ and $a^P_{\max} > a^E_{\max}$, and includes acceleration-aware state design and derivations for both levels and their coupling.

1 Problem Setup and Asymmetry Constraints

We consider a bounded 2D domain

$$\Omega = [-W, W]^2,$$

with simulation period Δt, primitive horizon $T_{\max}$, and capture radius r_c.

Pursuer and evader primitive states:

$$\boldsymbol{x}^P_t = (\boldsymbol{p}^P_t, \boldsymbol{v}^P_t, \boldsymbol{a}^P_t), \qquad \boldsymbol{x}^E_t = (\boldsymbol{p}^E_t, \boldsymbol{v}^E_t, \boldsymbol{a}^E_t, \psi^E_t).$$

Relative variables:

$$\boldsymbol{r}_t = \boldsymbol{p}^E_t - \boldsymbol{p}^P_t, \qquad d_t = \|\boldsymbol{r}_t\|, \qquad \Delta\boldsymbol{v}_t = \boldsymbol{v}^E_t - \boldsymbol{v}^P_t.$$

Asymmetric game constraints (required)

$$v^P_{\max} < v^E_{\max}, \qquad a^P_{\max} > a^E_{\max}. \tag{1}$$

This means capture must rely on *maneuverability/interception geometry*, not pure speed matching.

2 Primitive Dynamics Model

Both agents follow damped dynamics (project-consistent low-level physics abstraction):

$$M\dot{\boldsymbol{v}} = \boldsymbol{\tau} - D_l\boldsymbol{v} - D_q\|\boldsymbol{v}\|\boldsymbol{v}, \tag{2}$$

$$\boldsymbol{a} = \frac{1}{M}\left(\boldsymbol{\tau} - D_l\boldsymbol{v} - D_q\|\boldsymbol{v}\|\boldsymbol{v}\right), \quad \|\boldsymbol{a}\| \le a_{\max},\ \|\boldsymbol{v}\| \le v_{\max}. \tag{3}$$

Boundary projection/reflection is applied after integration.

3 Low-Level Continuous Control MDP (Execution Layer)

We define the low-level execution process as

$$\mathcal{M}_L = (\mathcal{S}_L, \mathcal{U}_L, P_L, R_L, \gamma_L),$$

where $\mathcal{U}_L \subset \mathbb{R}^m$ is *continuous*.

3.1 State design (acceleration-aware)

A sufficient low-level state for interception execution:

$$s_t^L = \left(\boldsymbol{p}_t^P, \boldsymbol{v}_t^P, \boldsymbol{a}_t^P, \boldsymbol{r}_t, \Delta\boldsymbol{v}_t, \boldsymbol{g}_t, \dot{\boldsymbol{g}}_t\right), \tag{4}$$

where $\boldsymbol{g}_t$ is high-level subgoal/virtual interception target.

Why acceleration must be observed Under (1), feasible capture depends on transient burst and turn response. Without $\boldsymbol{a}_t^P$ (or equivalent finite-difference surrogate), low-level policy cannot infer current thrust effectiveness and damping-limited maneuver phase, causing poor interception timing.

3.2 Continuous action parameterization (Option A)

Two equivalent realizations:

- **PID realization (current project-friendly):** low-level action is continuous command $u_t = [v_x^{des}, v_y^{des}, \psi^{des}]$ or $[F_x, F_y, M_z]$, then PID outputs thrust.
- **Continuous actor realization:** policy directly outputs bounded continuous control command u_t (e.g., SAC/TD3/PPO-Continuous), optionally filtered by safety/PID shell.

3.3 PID execution equations (project-aligned style)

With desired velocity command $\boldsymbol{v}_t^{des}$:

$$\boldsymbol{e}_t^v = \boldsymbol{v}_t^{des} - \boldsymbol{v}_t^P, \qquad \boldsymbol{f}_t = K_p^v \boldsymbol{e}_t^v - K_d^v \boldsymbol{v}_t^P, \tag{5}$$

plus attitude/yaw loop (if used):

$$M_{z,t} = K_p^\psi(\psi_t^{des} - \psi_t) + K_d^\psi(r_t^{des} - r_t). \tag{6}$$

Then actuator command $\boldsymbol{\tau}_t^P = \mathrm{sat}(\boldsymbol{f}_t, M_{z,t})$ is integrated by dynamics.

3.4 Low-level reward

Execution-level reward emphasizes controllability and goal tracking:

$$r_t^L = -w_g\|\boldsymbol{p}_t^P - \boldsymbol{g}_t\| - w_v\|\boldsymbol{v}_t^P - \boldsymbol{v}_t^{des}\| - w_u\|u_t\|^2 - w_j\|\boldsymbol{a}_t^P - \boldsymbol{a}_{t-1}^P\| + r_{safe}. \tag{7}$$

This explicitly regularizes acceleration transients and avoids unstable bang-bang behavior.

4 High-Level Pursuit MDP (Decision Layer)

High-level process (macro period K primitive steps):

$$\mathcal{M}_H = (\mathcal{S}_H, \mathcal{A}_H, P_H, R_H, \gamma_H).$$

4.1 State

Recommended high-level state:

$$s_t^H = \big(\boldsymbol{r}_t, \Delta\boldsymbol{v}_t, d_t, \hat{\boldsymbol{b}}_t, \hat{\boldsymbol{a}}_t^P, \hat{\boldsymbol{a}}_t^E\big), \tag{8}$$

where $\hat{\boldsymbol{b}}_t$ encodes boundary proximity, and acceleration terms may be smoothed estimates.

4.2 Action (discrete macro-intent)

$$\mathcal{A}_H = \{0, 1, \ldots, N_H - 1\}, \tag{9}$$

with semantics such as:

- $a = 0$: direct pursuit (aim at current evader position),
- $a \in \{1..8\}$: directional interception waypoint,
- optional extra intents: boundary herding, cut-off, brake-and-turn.

Action is mapped by $\Phi_H : a_t^H \mapsto (\boldsymbol{g}_t, \text{mode}_t)$, then handed to low-level controller for K steps.

4.3 High-level reward (task objective)

$$r_t^H = \alpha_1(d_t - d_{t+K}) + \alpha_2\, \overline{v}_{t:t+K}^{close} + \alpha_3\, \overline{r}_{t:t+K}^{trap} \tag{10}$$

$$- \alpha_4\, \overline{c}_{t:t+K}^{time} + \alpha_5\, \mathbb{I}[d_{t+K} < r_c]. \tag{11}$$

This captures interception progress and final capture.

5 Unified SMDP Coupling (High-Level + Low-Level)

Option A is naturally modeled as SMDP:

$$\mathcal{M}_S = (\mathcal{S}_H, \mathcal{A}_H, \mathcal{P}_S, \mathcal{R}_S, \gamma).$$

For each high-level decision a_t^H, low-level controller executes for random duration $\tau_t \in \{1, \ldots, K\}$ (early termination on capture/failure).

$$\mathcal{P}_S(s'|s,a) = \sum_{\tau=1}^{K} P(s', \tau | s, a), \tag{12}$$

$$\mathcal{R}_S(s,a) = \mathbb{E}\left[\sum_{i=0}^{\tau-1} \gamma^i r_{t+i} \mid s_t = s, a_t = a\right]. \tag{13}$$

Bellman equation:

$$Q(s,a) = \mathbb{E}\left[\sum_{i=0}^{\tau-1} \gamma^i r_{t+i} + \gamma^\tau \max_{a'} Q(s', a')\right]. \tag{14}$$

6 Non-asymmetric feasibility insight (why acceleration matters)

Under pure pursuit geometry, if pursuer speed is strictly lower, long-run capture is not guaranteed in open space. Capture becomes feasible by exploiting:

1. higher pursuer acceleration $a_{\max}^P > a_{\max}^E$,
2. finite turn-rate and damping of evader,
3. boundary-induced confinement and herding.

Hence acceleration-aware low-level execution is structurally necessary, not optional.

7 Practical design answers for your two concerns

7.1 Concern 1: "Do we need acceleration observation?"

Yes. At least one of the following must be included in low-level input:

- explicit (a_x, a_y),
- finite-difference estimate $(v_t - v_{t-1})/\Delta t$,
- short frame stack containing recent velocities/actions.

Best practice for this project: include explicit acceleration + 2–4 frame history.

7.2 Concern 2: "Is discrete low-level action suitable for acceleration tracking?"

Not ideal. Discrete low-level actions quantize thrust and degrade acceleration shaping. Option A solves this by:

- keeping **high-level** discrete (good for strategy),
- making **low-level** continuous (PID/continuous actor),
- preserving safety and controllability under constraints.

8 Optimization objectives

8.1 High-level optimization (discrete PPO)

$$\mathcal{L}^H_{clip} = \mathbb{E}\left[\min\left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \quad \rho_t = \frac{\pi_{\theta_H}(a^H_t|s^H_t)}{\pi_{\theta^{old}_H}(a^H_t|s^H_t)}. \tag{15}$$

8.2 Low-level optimization

If low-level is fixed PID, no policy optimization at this layer. If low-level is continuous RL actor $\pi_{\theta_L}(u|s^L)$, use SAC/TD3/PPO-Continuous objective over primitive steps with safety regularization.

9 Implementation mapping to current project files

- Decision-layer 2D formulation reference: `core/environment/low_level_env_2d_v2.py`
- Continuous control stack reference: `core/environment/low_level_env.py` + `core/control/dual_loop_pid`
- Pursuit-evasion asymmetry and acceleration-rich observation reference: `pursuit_evasion_2d/env_asymmetri`
- HRL temporal abstraction reference: `pursuit_evasion_2d/train_hrl_pe.py`
- Stage-1 continuous low-level training: `pursuit_evasion_2d/env_low_level_target_continuous.py` + `train_low_level_continuous.py`
- Stage-2 high-level pursuit with PID execution: `pursuit_evasion_2d/env_asymmetric_pe_v2_pid_option_a.p` + `train_hrl_option_a_pid.py`

Remark 1. *This document specifies Option A as a practical two-stage pipeline: (Stage 1) train continuous low-level goal-reaching controller; (Stage 2) freeze/retain PID execution interface and train high-level discrete pursuit policy on induced SMDP.*

Compile

```
cd docs
pdflatex hrl_mdp_complete.tex
```